Lecture 1: Introduction to Computer Systems & Data Representation

1. What is Computer Architecture?

Computer architecture is fundamental to understanding how computers work. It's often broken down into two closely related, but distinct, concepts:

- **Computer Architecture:** This refers to the **attributes of a system visible to a programmer**. It's concerned with the logical and functional aspects of a computer system. Think of it as the *instruction set design*, the *number of bits used for data*, *I/O mechanisms*, and *addressing techniques*. If a feature impacts how a program is written (e.g., specific instructions, register availability), it's part of the architecture. It defines *what* the computer does.
- **Computer Organization:** This refers to the **physical implementation of the computer architecture**. It deals with the operational units and their interconnections that realize the architectural specifications. This includes things like the *control signals*, *interfaces between the computer and peripherals*, and the *memory technology used*. It defines *how* the computer does it.

Analogy: Imagine a car.

- **Architecture** is the design of the driver's interface: the steering wheel, pedals, gear stick, dashboard layout. These are the "programmer-visible" attributes.
- **Organization** is the engine design, transmission, braking system, and all the internal components that make the car function according to the interface design.

In essence: Architecture is the "blueprint," while Organization is the "construction." A single architecture can have many different organizations (e.g., Intel's x86 architecture has been implemented in many different generations of processors with vastly different internal organizations).

2. Brief History of Computers

The evolution of computers is a fascinating journey, marked by significant technological breakthroughs:

- **Early Mechanical Computers (17th - 19th Century):**
    - **Blaise Pascal (1642):** Invented the Pascaline, an arithmetic machine capable of addition and subtraction.
    - **Gottfried Leibniz (1673):** Developed the Stepped Reckoner, which could also multiply and divide.
    - **Charles Babbage (1800s):** Designed the Difference Engine (for polynomial calculations) and the Analytical Engine (considered the first design of a general-purpose computer, including an ALU, control flow, and memory). Ada

Lovelace, Lord Byron's daughter, wrote programs for Babbage's Analytical Engine, making her often cited as the first computer programmer.

- **Electro-Mechanical Computers (Early 20th Century):**
  - **Konrad Zuse (1930s-1940s):** Built the Z1, Z2, Z3 (first functional program-controlled computer) in Germany.
  - **Howard Aiken (1940s):** Developed the Harvard Mark I with IBM.
- **First Generation: Vacuum Tubes (1940s-1950s)**
  - **ENIAC (Electronic Numerical Integrator and Computer, 1946):** First large-scale electronic digital computer. Used thousands of vacuum tubes, enormous, consumed vast power, and generated immense heat. Programmed by rewiring.
  - **UNIVAC I (Universal Automatic Computer, 1951):** First commercial computer.
  - **Key Characteristics:** Bulky, slow, expensive, high power consumption, used machine language.
- **Second Generation: Transistors (1950s-1960s)**
  - **Invention of Transistor (Bell Labs, 1947):** Replaced vacuum tubes. Smaller, faster, more reliable, less heat, less power.
  - **Introduction of Assembly Language and early High-Level Languages (FORTRAN, COBOL).**
  - **Key Characteristics:** Smaller, faster, more reliable, lower power, magnetic cores for memory.
- **Third Generation: Integrated Circuits (ICs) (1960s-1970s)**
  - **Invention of IC (Jack Kilby & Robert Noyce, late 1950s):** Miniaturized and integrated multiple transistors onto a single silicon chip.
  - **Introduction of Operating Systems.**
  - **Key Characteristics:** Smaller still, faster, more reliable, mass production, emergence of minicomputers.
- **Fourth Generation: Microprocessors (1970s-Present)**
  - **Invention of Microprocessor (Intel 4004, 1971):** A single chip containing the entire CPU. Led to personal computers (PCs).
  - **Very Large Scale Integration (VLSI):** Millions of transistors on a single chip.
  - **Graphical User Interfaces (GUIs), networking, the Internet.**

    - **Key Characteristics:** Desktop PCs, laptops, workstations, the internet, highly scalable, very high performance.

- **Fifth Generation: AI & Parallel Processing (Present & Beyond)**
    - Focus on parallel processing, artificial intelligence, natural language processing, machine learning, quantum computing.
    - **Key Characteristics:** Supercomputers, AI systems, highly parallel architectures, distributed computing.

3. Components of a Computer System: Hardware vs. Software

A computer system is a complex integration of two main components:

3.1. Hardware

Hardware refers to the physical, tangible components of a computer system. These are the parts you can see and touch.

- **Central Processing Unit (CPU):** The "brain" of the computer. It executes instructions, performs arithmetic operations, and manages the flow of data.
    - **Arithmetic Logic Unit (ALU):** Performs arithmetic (addition, subtraction) and logical (AND, OR, NOT) operations.
    - **Control Unit (CU):** Manages and coordinates the activities of all other components by sending control signals. It fetches instructions, decodes them, and directs operations.
    - **Registers:** Small, high-speed storage locations within the CPU used to hold data temporarily during processing.
- **Memory (Main Memory/RAM):**
    - **Random Access Memory (RAM):** Volatile memory used to temporarily store data and programs that the CPU is currently using. It's fast but loses its contents when the power is off.
    - **Read-Only Memory (ROM):** Non-volatile memory that stores essential boot-up instructions (BIOS/UEFI) that cannot be easily altered.
- **Storage Devices (Secondary Storage):** Non-volatile memory used for long-term storage of data and programs.
    - **Hard Disk Drives (HDDs):** Traditional mechanical storage, magnetic platters.
    - **Solid State Drives (SSDs):** Faster, more durable, flash-based storage.
    - **Optical Drives:** CDs, DVDs, Blu-ray discs.
    - **USB Flash Drives:** Portable, flash-based.

- **Input Devices:** Allow users to enter data and commands into the computer.
    - Keyboard, Mouse, Microphone, Scanner, Webcam, Touchscreen.
- **Output Devices:** Display or present the results of processing to the user.
    - Monitor/Display, Printer, Speakers, Projector.
- **Communication Devices:** Facilitate data exchange with other computers or networks.
    - Network Interface Card (NIC), Modem, Wi-Fi Adapter.
- **Buses:** Pathways that connect different components of the computer, allowing them to communicate.
    - **Data Bus:** Carries data.
    - **Address Bus:** Carries memory addresses.
    - **Control Bus:** Carries control signals (e.g., read/write commands).

3.2. Software

Software refers to the set of instructions, data, or programs used to operate computers and execute specific tasks. It is intangible.

- **System Software:**
    - **Operating System (OS):** Manages computer hardware and software resources, provides common services for computer programs. Examples: Windows, macOS, Linux, Android, iOS.
    - **Device Drivers:** Programs that allow the operating system to communicate with specific hardware devices.
    - **Utility Software:** Programs that perform maintenance and support tasks (e.g., antivirus, disk defragmenter, backup tools).
- **Application Software:**
    - Programs designed to perform specific tasks for the user. Examples: Word processors (MS Word), web browsers (Chrome, Firefox), media players (VLC), games, accounting software, graphic design software.

4. Data Representation

Computers operate on electrical signals, which can be in one of two states: ON or OFF. These states are represented by **binary digits (bits)**: 0 (OFF) and 1 (ON). All data processed by a computer, whether it's text, numbers, images, or sound, must be converted into this binary format.

4.1. Number Systems

Different number systems are used to represent quantities. We are most familiar with the decimal system.

- **Decimal System (Base-10):**
  - Uses 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
  - Each digit's position represents a power of 10.
  - Example: $123_{10}=(1\times10^2)+(2\times10^1)+(3\times10^0)=100+20+3=123$
- **Binary System (Base-2):**
  - Uses 2 digits: 0, 1.
  - Each digit's position represents a power of 2.
  - Example: $1011_2=(1\times2^3)+(0\times2^2)+(1\times2^1)+(1\times2^0)=8+0+2+1=11_{10}$
- **Octal System (Base-8):**
  - Uses 8 digits: 0, 1, 2, 3, 4, 5, 6, 7.
  - Each digit's position represents a power of 8.
  - Useful as a shorthand for binary (3 binary digits = 1 octal digit).
  - Example: $23_8=(2\times8^1)+(3\times8^0)=16+3=19_{10}$
- **Hexadecimal System (Base-16):**
  - Uses 16 symbols: 0-9 and A, B, C, D, E, F (where A=10, B=11, ..., F=15).
  - Each digit's position represents a power of 16.
  - Widely used in computing as a shorthand for binary (4 binary digits = 1 hexadecimal digit).
  - Example: $1A_{16}=(1\times16^1)+(10\times16^0)=16+10=26_{10}$

Conversions:

- **Any Base to Decimal:** Multiply each digit by its base raised to the power of its position (starting from 0 for the rightmost digit) and sum the results.

- **Decimal to Any Base:** Repeated division by the target base, collecting the remainders in reverse order.
    - **Example: Convert 26_{10} to Binary:** 26÷2=13 remainder 0 13÷2=6 remainder 1 6÷2=3 remainder 0 3÷2=1 remainder 1 1÷2=0 remainder 1 Reading remainders bottom-up: 11010_2

- **Binary to Octal/Hexadecimal:**
    - **Binary to Octal:** Group binary digits into sets of 3 from the right. Convert each group to its octal equivalent. Add leading zeros if necessary.
        - Example: $11010_2 \rightarrow 011\ 010_2 \rightarrow 3\ 2_8 = 32_8$
    - **Binary to Hexadecimal:** Group binary digits into sets of 4 from the right. Convert each group to its hexadecimal equivalent. Add leading zeros if necessary.
        - Example: $11010_2 \rightarrow 0001\ 1010_2 \rightarrow 1\ A_{16} = 1A_{16}$
- **Octal/Hexadecimal to Binary:** Convert each octal/hexadecimal digit into its 3-bit/4-bit binary equivalent.

4.2. Signed and Unsigned Integer Representation

Computers need a way to represent both positive and negative whole numbers.

- **Unsigned Integers:** Represent only non-negative (zero or positive) values. All bits are used to represent the magnitude.
    - An n-bit unsigned integer can represent values from 0 to $2^n - 1$.
    - Example: Using 4 bits, unsigned can represent $0(0000_2)$ to $15(1111_2)$.
- **Signed Integers:** Represent both positive and negative values. The leftmost bit (Most Significant Bit - MSB) is usually designated as the **sign bit**:
    - 0 for positive numbers
    - 1 for negative numbers

There are three common methods for representing signed integers:

3. **Sign-Magnitude Representation:**
    - The MSB is the sign bit. The remaining bits represent the magnitude (absolute value).
    - **Disadvantages:**
        - Two representations for zero (+0 and -0).
        - Arithmetic operations are complex.
    - Example (4 bits):
        - $0011_2 = +3$
        - $1011_2 = -3$
        - $0000_2 = +0$

- $1000_2 = -0$

4. **One's Complement Representation:**
    - Positive numbers are represented as in sign-magnitude.
    - Negative numbers are obtained by **inverting all the bits** (0s become 1s, and 1s become 0s) of the positive equivalent.
    - **Disadvantages:**
        - Still has two representations for zero (+0 and -0).
        - Arithmetic is slightly simpler than sign-magnitude but still problematic (end-around carry for addition).
    - Example (4 bits):
        - $0011_2 = +3$
        - To get -3: Invert $0011 \rightarrow 1100_2$
        - $0000_2 = +0$
        - $1111_2 = -0$

5. **Two's Complement Representation (Most Common):**
    - Positive numbers are represented as in sign-magnitude.
    - Negative numbers are obtained by:

1. Inverting all bits of the positive equivalent (one's complement).
2. Adding 1 to the result.

    - **Advantages:**
        - Only one representation for zero (000...02).
        - Arithmetic operations (addition, subtraction) are straightforward and uniform for both positive and negative numbers.
        - An n-bit two's complement number can represent values from $-(2^{n-1})$ to $+(2^{n-1}-1)$.
    - Example (4 bits):
        - $0011_2 = +3$
        - To get -3:
            1. Invert $0011 \rightarrow 1100$ (one's complement)
            2. Add $1 \rightarrow 1100+1 = 1101_2$. So, $1101_2 = -3$.

- $0000_2=0$ (unique zero)
- Range for 4 bits: $-(2^{4-1})$ to $+(2^{4-1}-1)=-8$ to $+7$.

Why Two's Complement is Important: It simplifies hardware design for arithmetic operations because addition circuits can be used for subtraction by adding the two's complement of the subtrahend.

4.3. Floating-Point Representation (IEEE 754 Standard - Single Precision Introduction)

To represent real numbers (numbers with fractional parts, e.g., 3.14, -0.005), computers use floating-point notation, similar to scientific notation. The **IEEE 754 standard** is the most widely used.

A floating-point number is generally represented as:
$(-1)^{sign} \times \text{Mantissa (or Significand)} \times \text{Base}^{\text{Exponent}}$

For the IEEE 754 Single Precision (32-bit) standard:

- **1 bit for Sign (S):** 0 for positive, 1 for negative.
- **8 bits for Exponent (E):** Stored in a biased form (e.g., for single precision, a bias of 127 is added to the actual exponent). This allows for easy comparison of floating-point numbers.
- **23 bits for Fraction (F) / Significand:** Represents the fractional part of the mantissa. The leading '1' before the decimal point is implicit (hidden bit) for normalized numbers, effectively giving 24 bits of precision.

General Structure (32-bit Single Precision):

S | EEEEEEEE | FFFFFFFFFFFFFFFFFFFFFFF

1 bit | 8 bits | 23 bits

Example (Conceptual - simplified): Let's say we want to represent 6.2510:

1. **Convert to Binary:** $6.25_{10}=110.01_2$
2. **Normalize (move decimal point):** $1.1001_2 \times 2^2$
3. **Identify Components:**
   - **Sign (S):** Positive, so S = 0.
   - **Exponent (E):** The actual exponent is 2. Add bias (127 for single precision): $2+127=129_{10}$. Convert 129 to 8-bit binary: 10000001_2.
   - **Fraction (F):** The part after the binary point in the normalized form, ignoring the leading 1: 1001_2. Pad with zeros to 23 bits: 10010000000000000000002.
4. **Assemble:** 0 | 10000001 | 10010000000000000000000

Key aspects of IEEE 754:

- Provides a standardized way to represent and perform arithmetic on real numbers.
- Handles special values like infinity (positive and negative) and "Not a Number" (NaN) for undefined results.
- **Trade-off:** Floating-point numbers offer a wider range of values but suffer from potential precision issues due to limited bits for the mantissa. This is why small errors can accumulate in complex calculations.

4.4. Character Representation (ASCII, Unicode - Brief Overview)

Computers need to represent not just numbers, but also text characters (letters, symbols, punctuation). This is done by assigning a unique binary code to each character.

- **ASCII (American Standard Code for Information Interchange):**
    - An early and widely adopted standard.
    - Uses 7 bits to represent 128 characters (0 to 127).
    - Includes uppercase and lowercase English letters, digits (0-9), punctuation, and control characters (e.g., Newline, Tab).
    - Example: 'A' is 6510 (010000012), 'a' is 9710 (01100001_2), '0' is 4810 (00110000_2).
    - **Extended ASCII:** Often uses 8 bits to include 128 more characters, but these extended characters vary depending on the specific "code page" used (e.g., for different European languages). This led to inconsistencies.
- **Unicode:**
    - Developed to overcome the limitations of ASCII (especially for representing characters from various languages globally).
    - Aims to provide a unique number for every character, no matter what platform, program, or language.
    - Supports over 1 million characters.
    - **Encodings (how Unicode characters are stored as bytes):**
        - **UTF-8 (Unicode Transformation Format - 8-bit):** The most common encoding on the web. It is a variable-width encoding, meaning characters can take 1 to 4 bytes. ASCII characters are represented using 1 byte (making it backward compatible with ASCII).
        - **UTF-16:** Uses 2 or 4 bytes per character.
        - **UTF-32:** Uses a fixed 4 bytes per character.